

# CMU-CS 462: Software Measurement and Analysis

Spring 2025-2026

Nguyen Duc Man

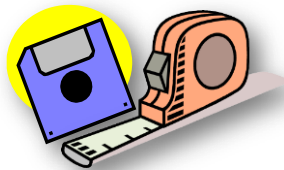
E: [mannd@duytan.edu.vn](mailto:mannd@duytan.edu.vn)

T: 0904235945

12/01/2026

1

## Measuring Internal Product Attributes: Software Size (cont'd)



12/01/2026

2

2

**1. (0.2 points)**

In size metrics, metrics are developed based on the \_\_\_\_\_

- a. number of Functions
- b. number of user inputs
- c. number of lines of code
- d. amount of memory usage

**1. (0.2 points)**

Which of the following are information domains required for determining function point in FPA (nhiều phương án)?

- a. Number of user Input
- b. Number of user Inquiries
- c. Number of external Interfaces
- d. Number of errors

12/01/2026

3

3

**1. (0.2 points)**

How to measure the defect density of a system (more than one)?

- a. defects per KLOC
- b. defects per function point
- c. total number of defects
- d. None of above

**1. (0.2 points)**

Which statement is not a measurement of Length in LOC?

- a. Count of physical lines including blank lines.
- b. Count of all Operators
- c. Count of all lines except blank lines and comments.
- d. Count of all statements except comments

12/01/2026

4

4

## Contents

1. Software size
2. Size: Length (code, specification, design)
3. Size: Reuse
4. Size: Functionality (function point, feature point, object point, use-case point)
5. Size: Complexity

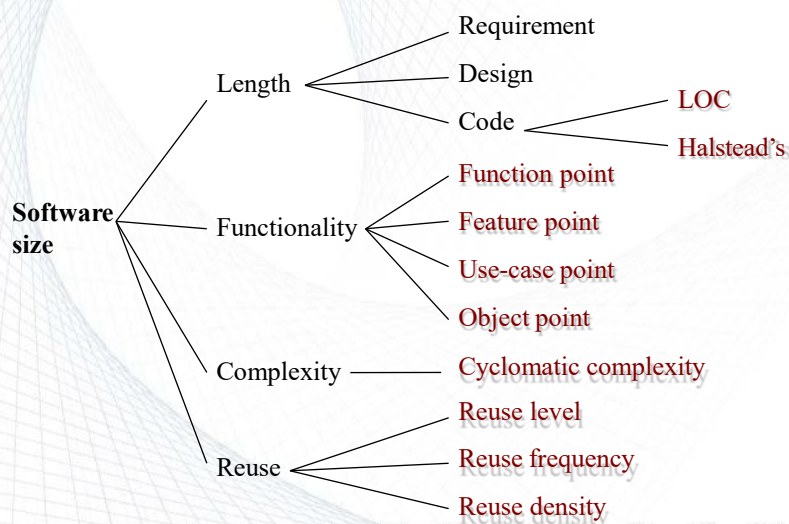


12/01/2026

5

5

## Software Size Metrics: Summary



12/01/2026

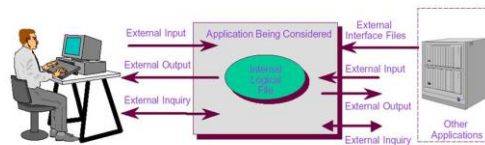
6

6

## Function Points (IFPUG)

### Example 1

#### Function Points (FPA) Software Model



To identify : EI-EO-EQ & ILF-EIF

12/01/2026

7

7

## FP Example /1a

### Specification: Spell Checker

- Checks all words in a document by comparing them to a list of words in the internal dictionary and an optional user-defined dictionary
- After processing the document sends a report on all misspelled words to standard output
- On request from user shows number of words processed on standard output
- On request from user shows number of spelling errors detected on standard output
- Requests can be issued at any point in time while processing the document file



12/01/2026

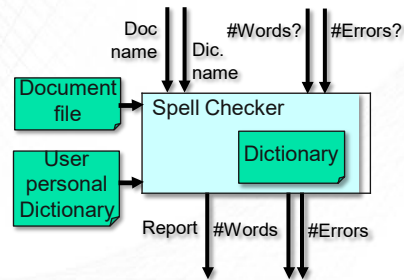
8

8

## FP Example /1b

### Spell Checker

- Convert spec to a diagram



#### Specification:

- Checks all words in a document by comparing them to a list of words in the internal dictionary and an optional user-defined dictionary
- After processing the document sends a report on all misspelled words to standard output
- On request from user shows number of words processed on standard output
- On request from user shows number of spelling errors detected on standard output
- Requests can be issued at any point in time while processing the document file

12/01/2026

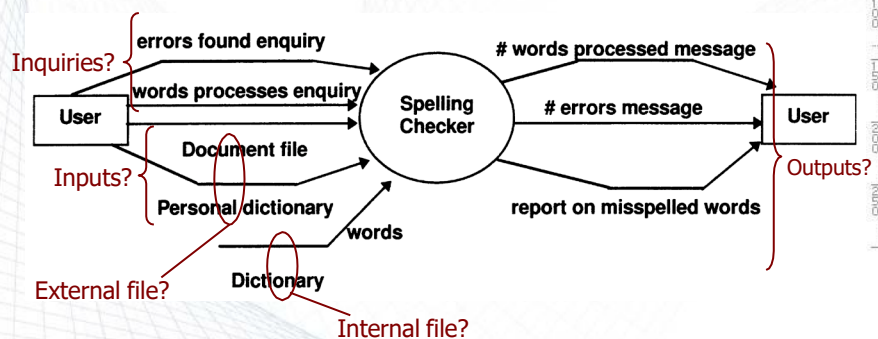
9

9

## FP Example /1c

### Spell Checker

- Convert spec to a diagram



12/01/2026

10

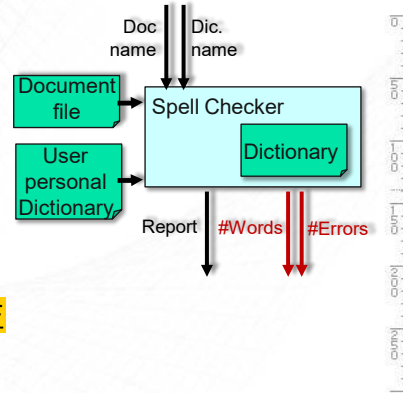
10



## FP Example /1d

### Solution A:

- EI: Doc. name + User Dic. name → 2
- EO: Report + #Words + #Errors → 3
- EQ: --
- EIF: Document + User Dictionary → 2
- ILF: Dictionary → 1



**EI EO EQ EIF ILF**

12/01/2026

11

11

## FP Example /1e

| Element                        | Weighting Factor |         |         |
|--------------------------------|------------------|---------|---------|
|                                | Simple           | Average | Complex |
| External inputs (EI)           | 3                | 4       | 6       |
| External outputs (EO)          | 4                | 5       | 7       |
| External inquiries (EQ)        | 3                | 4       | 6       |
| External interface files (EIF) | 5                | 7       | 10      |
| Internal logical files (ILF)   | 7                | 10      | 15      |

- $N_{EI}=2$  (number of EI): document name, user-defined dictionary name
- $N_{EO}=3$  (number of EO): misspelled word report, number-of-words-processed message, number-of-errors-so-far message
- $N_{EQ}=0$  (number of EQ): --
- $N_{EIF}=2$  (number of EIF): document file, personal dictionary
- $N_{ILF}=1$  (number of ILF): dictionary

12/01/2026

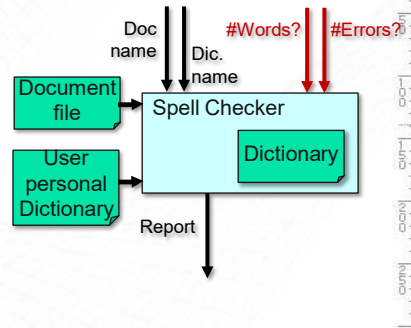
12

12

## FP Example /1f

### Solution B:

- EI: Doc. name + Dic. name → 2
- EO: Report → 1
- EQ: #Words? + #Errors? → 2
- EIF: Document + User Dictionary → 2
- ILF: Dictionary → 1



EI EO EQ EIF ILF

12/01/2026

13

13

## FP Example /1g

| Element                        | Weighting Factor |         |         |
|--------------------------------|------------------|---------|---------|
|                                | Simple           | Average | Complex |
| External inputs (EI)           | 3                | 4       | 6       |
| External outputs (EO)          | 4                | 5       | 7       |
| External inquiries (EQ)        | 3                | 4       | 6       |
| External interface files (EIF) | 5                | 7       | 10      |
| Internal logical files (ILF)   | 7                | 10      | 15      |

- $N_{EI}=2$  (number of EI): document name, user-defined dictionary name
- $N_{EO}=1$  (number of EO): misspelled word report
- $N_{EQ}=2$  (number of EQ): number-of-words-processed message, number-of-errors-so-far message
- $N_{EIF}=2$  (number of EIF): document file, personal dictionary
- $N_{ILF}=1$  (number of ILF): dictionary

12/01/2026

14

14

## FP Example /1h

- Suppose that

$$F_1=F_2=F_6=F_7=F_8=F_{14}=3;$$

$$F_4=F_{10}=5$$

and the rest are zero.

$$VAF = 0.65 + 0.01 \sum_{j=1}^{14} F_j$$

Solution A:

Solution B:

12/01/2026

15

15

## FP Example /1i –Discussion

Which solution is correct?

- Solution A?
- Solution B?
- None of them? (cf. Textbook)

**Answer: Solution A is correct!**

**Why? The difference is because of correct interpretation of EO and EQ for the requirement #5**



Wrong  
Calculation in  
Fenton's book

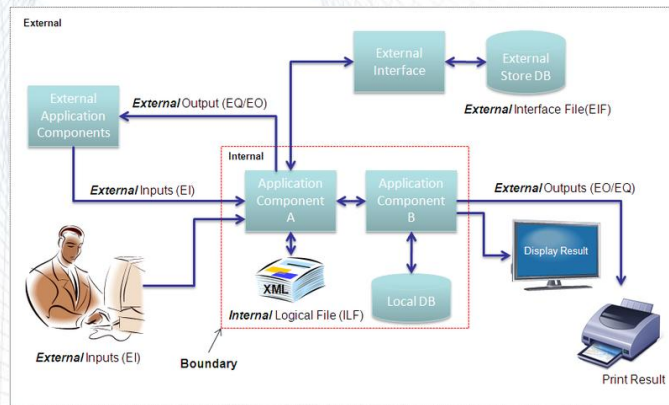
12/01/2026

16

16



## Function Points (IFPUG)



### Example 2

12/01/2026

17

17

## FP: Example /2a

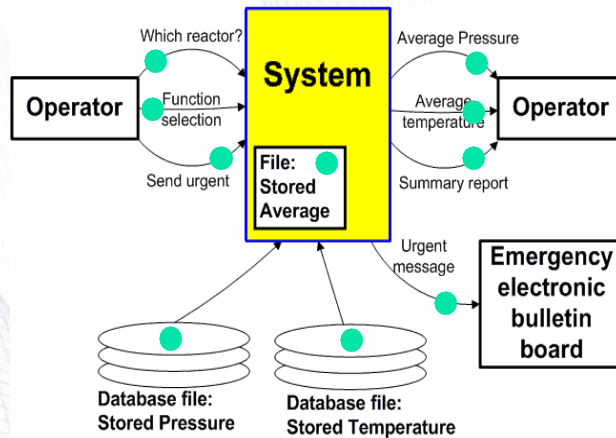
- Consider the following system that processes various commands from the operator of a chemical plant. Various interactions of the system with the operator and internal and external files are shown.
- The system consults two databases for stored pressure and temperature readings. In the case of emergency, the system asks the user whether to send the results to an electronic emergency bulletin board or not.
- Calculate the unadjusted function point count (UFC) for this system.

12/01/2026

18

18

## FP: Example /2b



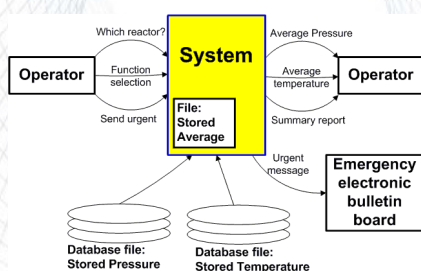
EI EO EQ EIF ILF

12/01/2026

19

19

## FP: Example /2c



- $N_{EI}$  number of external inputs 2
- $N_{EO}$  number of external outputs 3
- $N_{EQ}$  number of external inquiries 1
- $N_{EIF}$  number of external interface files 2
- $N_{ILF}$  number of internal logical files 1

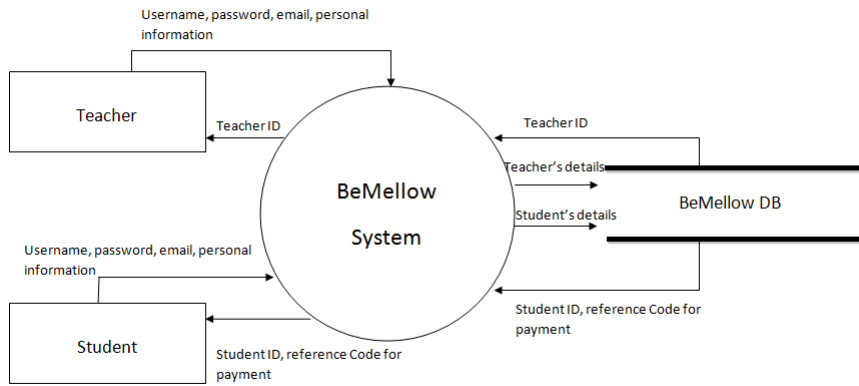
$$UFC = 4N_{EI} + 5N_{EO} + 4N_{EQ} + 7N_{EIF} + 10N_{ILF}$$

$$UFC = 4 \times 2 + 5 \times 3 + 4 \times 1 + 7 \times 2 + 10 \times 1 = 51$$

12/01/2026

20

20



12/01/2026

21

21

## Review: Software Size

- Size measurement must reflect *effort*, *cost* and *productivity*.
- Defining software size in terms of *length*, *functionality*, and *complexity*, each capturing a key aspect of software size.
- Basic measure for length is LOC.
- Basic measure for functionality is Function Point (FP).
- FP is computed in two steps:
  - 1) Calculating an *Unadjusted Function point Count (UFC)*.
  - 2) Multiplying the UFC by a *Value Adjustment Factor (VAF)*.

The final (adjusted) Function Point is:

$$FP = UFC \times VAF$$

- FP variations: feature point, object point, use-case point.

12/01/2026

22

22

## FP vs. LOC /1

- A number of studies have attempted to relate LOC and FP metrics (Jones, 1996).
- The average number of source code statements per function point has been derived from case studies for numerous programming languages.
- Languages have been classified into different levels according to the relationship between LOC and FP.

12/01/2026

23

23

## FP vs. LOC /2

| Language             | Typical SLOC per FP |
|----------------------|---------------------|
| 1GL Default Language | 320                 |
| 2GL Default Language | 107                 |
| 3GL Default Language | 80                  |
| 4GL Default Language | 20                  |
| Code generators      | 15                  |
| Assembler            | 320                 |
| C                    | 148                 |
| Basic                | 107                 |
| Pascal               | 90                  |

12/01/2026

24

24

# FP vs. LOC /3

| Language     | Typical SLOC per FP |
|--------------|---------------------|
| C#           | 59                  |
| C++          | 60                  |
| PL/SQL       | 46                  |
| Java 2       | 60                  |
| Visual Basic | 50                  |
| Delphi       | 18                  |
| HTML 4       | 14                  |
| SQL          | 13                  |
| Excel        | 47                  |

Newer Data!

<http://www.qsm.com/FPgearing.html>

12/01/2026

25

25

```
01 DATA SEGMENT
02     NUM1 DB 9H
03     NUM2 DB 7H
04     RESULT DB ?
05 ENDS
06
07 CODE SEGMENT
08     ASSUME DS:DATA CS:CODE
09 START:
10     MOV AX,DATA
11     MOV DS,AX
12
13     MOV AL,NUM1
14     ADD AL,NUM2
15
16     MOV RESULT,AL
17
18     MOV AH,4CH
19     INT 21H
20 ENDS
21 END START
22
```

12/01/2026

26

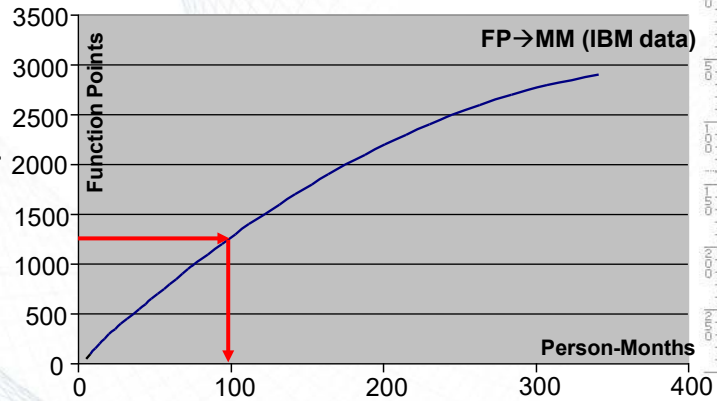
26





## Effort Estimation with FP

Effort estimation based on organization-specific data from past projects.



12/01/2026

27

27



## FP: Advantages –Summary

- Can be counted before design or code documents exist
- Can be used for estimating project cost, effort, schedule early in the project life-cycle
- Helps with contract negotiations
- Is standardized (though several competing standards exist)

12/01/2026

28

28

## FP: Critics

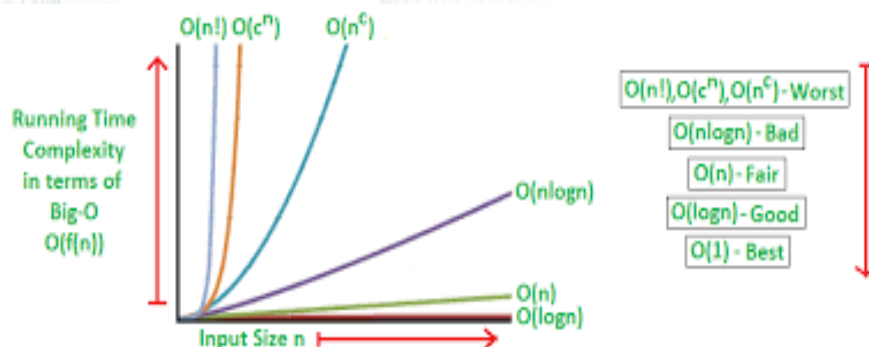
- FP is a subjective measure: affected by the selection of weights by external users.
- Function point calculation requires a full software system specification. It is therefore difficult to use function points very early in the software development lifecycle.
- Physical meaning of the basic unit of FP is unclear.
- Unable to account for new versions of I/O, such as data streams, intelligent message passing, etc.
- Not suitable for “complex” software, e.g., real-time and embedded applications.

12/01/2026

29

29

## Feature Point



12/01/2026

30

30

## Feature Point /1

- Function points were originally designed to be applied to business information systems. Extensions have been suggested called *feature points* which may enable this measure to be applied to other software engineering applications.
- Feature points accommodate applications in which the “*algorithmic complexity*” is high such as real-time, process control, and embedded software applications.
- For conventional software and information systems functions and feature points produce similar results. For complex systems, feature points often produce counts about %20~%35 higher than function points.

12/01/2026

31

31

## Feature Point /2

- Feature points are calculated the same way as FPs with the additions of *algorithms* as an additional software characteristic.
- Counts are made for the five FP categories, i.e., number of external inputs, external outputs, inquiries, internal files, external interfaces, plus:
  - **Algorithm ( $N_a$ ):** A bounded computational problem such as inverting a matrix, decoding a bit string, or handling an interrupt.

12/01/2026

32

32

## Feature Point /3

- Feature points are calculated using:
  - Number of external inputs  $\times 4$
  - Number of external outputs  $\times 5$
  - Number of external inquiries  $\times 4$
  - Number of external interface files  $\times 7 \leftarrow$
  - Number of internal files  $\times 10$
  - Algorithms  $\times 3 \leftarrow$

$$UF_eC = 4N_{EI} + 5N_{EO} + 4N_{EQ} + 7N_{EIF} + 10N_{ILF} + 3N_{A}$$

- The  $UF_eC$  used in the function point calculation to calculate the feature points.

12/01/2026

33

33

## Object Point /1

- Object points are used as an initial measure for size way early in the development cycle, during feasibility studies.
- An initial size measure is determined by counting the number of *screens, reports*, and third-generation *components* that will be used in the application.
- Each object is classified as *simple, medium*, or *difficult*.

12/01/2026

34

34

## Object Point /2

- Object point complexity levels for **screens**.

| Number of views contained | Number and source of data tables     |                                        |                                      |
|---------------------------|--------------------------------------|----------------------------------------|--------------------------------------|
|                           | Total <4<br><2 servers<br><2 clients | Total <8<br>2-3 servers<br>3-5 clients | Total 8+<br>>3 servers<br>>5 clients |
| < 3                       | Simple                               | Simple                                 | Medium                               |
| 3 - 7                     | Simple                               | Medium                                 | Difficult                            |
| 8+                        | Medium                               | Difficult                              | Difficult                            |

12/01/2026

35

35

## Object Point /3

- Object point complexity levels for **reports**.

| Number of views contained | Number and source of data tables     |                                        |                                      |
|---------------------------|--------------------------------------|----------------------------------------|--------------------------------------|
|                           | Total <4<br><2 servers<br><2 clients | Total <8<br>2-3 servers<br>3-5 clients | Total 8+<br>>3 servers<br>>5 clients |
| 0 - 1                     | Simple                               | Simple                                 | Medium                               |
| 2 - 3                     | Simple                               | Medium                                 | Difficult                            |
| 4+                        | Medium                               | Difficult                              | Difficult                            |

12/01/2026

36

36



## Object Point /3

- The number in each cell is then weighted and summed to get the object point.
- The weights represent the relative effort required to implement an instance of that complexity level.
- **Complexity level for object point:**

| Object Type   | Simple | Medium | Difficult |
|---------------|--------|--------|-----------|
| Screen        | 1      | 2      | 3         |
| Report        | 2      | 5      | 8         |
| 3GL component | -      | -      | 10        |

*New object points = (object points)  $\times$  (100 - r) / 100*  
 Assuming that % r of the objects will be reused from previous projects.

12/01/2026

37

37

## How to Calculate OP?

**Work in pairs (thảo luận theo nhóm 2)!**

12/01/2026

38

38

## Effort Estimation with OP

- Formula:

$$\text{Effort [Person-Month]} = \text{OP} / \text{PROD}$$

- How to measure PROD?

| Developer Experience/Skills | Very Low | Low | Average (Nominal) | High | Very High |
|-----------------------------|----------|-----|-------------------|------|-----------|
| PROD                        | 4        | 7   | 13                | 25   | 50        |

- Example:

$$\text{Effort [Person-Month]} = 13 / 13 = 1$$

Assuming average experience and 0% reuse.

12/01/2026

39

39

## Example: OP

- Suppose that you have
  - 4 screens, 2 of them simple (weight 1) and 2 of them medium (weight 2)
  - 3 reports, 2 of them simple (weight 2) and 1 medium (weight 5),

then the total number of OP is:

$$\text{OP} = 2 \times 1 + 1 \times 2 + 2 \times 2 + 1 \times 5 = 13$$

- If you have any acquired component, give it the weight 10 and add it to the OP.
- If you have any part of your system reused from a previous generation, define a percentage of reuse (say 10%) and then adjust the value of OP accordingly.

$$\text{OP}_{\text{new}} = 13 \times (100-10)/100 = 11.7$$

12/01/2026

40

40

## Example 2

- Determine object points for a smart vending machine assuming that the system is average size (total number of clients-servers is less than 8) using the following data:

| <i>Screens</i>               | <i>Views (Data Items)</i>                   |
|------------------------------|---------------------------------------------|
| <i>User Screens</i>          |                                             |
| S1. Buying Screen            | Amount of coins to insert, selection        |
| <i>Administrator Screens</i> |                                             |
| S2. Inventory Update         | Inventory input update (20 data items)      |
| S3. Change Update            | Change infrared input update (4 data items) |

12/01/2026

41

41

## Example 2 (cont'd)

| <i>Reports</i>               | <i>Views (Data Items)</i>                                    |
|------------------------------|--------------------------------------------------------------|
| <i>User Reports</i>          |                                                              |
| R1. Display money entered    | Amount of money entered                                      |
| R2. Not enough money entered | Insufficient funds                                           |
| R3. Returning change         | Amount of change                                             |
| R4. Out of stock message     | Out of stock                                                 |
| R5. No change message        | No change                                                    |
| R6. Selection Approved       | Release selection signal                                     |
| <i>Administrator Reports</i> |                                                              |
| R7. Machine Report           | Inventory (20 data items), change information (4 data items) |

12/01/2026

42

42

## Example 2 (cont'd)

| <i>Simple</i>            | <i>Medium</i> | <i>Difficult</i> |
|--------------------------|---------------|------------------|
| <i>S1</i>                | <i>S3</i>     | <i>S2</i>        |
| <i>R1 R2 R3 R4 R5 R6</i> | <i>None</i>   | <i>R7</i>        |

$$OP = (1 \times 1) + (1 \times 2) + (1 \times 3) + (2 \times 6) + (5 \times 0) + (8 \times 1)$$

$$OP = 26$$

$$Effort = 26/13 = 2 \text{ person month}$$

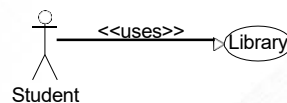
12/01/2026

43

43

## Use-Case Point /1

- *Function Point* is a method to measure software size from a *requirements* perspective.
- *Use-Case* is a method to develop *requirements*. Use Cases are used to validate a proposed design and to ensure it meets all requirements.



**Question:** How to use Use-Cases to measure function point and vice-versa?

12/01/2026

44

44

## Use-Case Point /2

**Question:** How to use Use-Cases to measure function point and vice-versa?

Two methods:

1. Identify and weight *actors* and *use-cases*
2. Count the inputs, outputs, files and data inquiries from use-cases (using the *use-case definition* and *activity diagram*).

12/01/2026

45

45

## Use Case Point /3

- Use Case Point Estimation has its origin in the work by Gustav Karner (1993)
- Process:
  1. Identify and weight Actors (→ UAW)
  2. Identify and weight Use Cases (→ UUCW)
  3. Calculate Unadjusted Use Case Points (→ UUCP = UAW + UUCW)
  4. Calculate Value Adjustment Factor (→ VAF)
  5. Calculate (Adjusted) Use Case Points  
→  $UCP = UUCP * VAF$

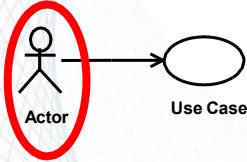
12/01/2026

46

46



## Use Case Point: Actors



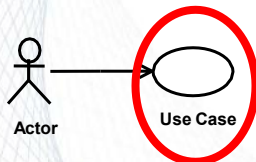
- For each Actor, determine whether the actor is simple, average or complex.
- A **simple actor** represents another system with a defined Application Programming Interface (API). [weight: 1]
- An **average actor** is either another system that interacts through a protocol (HTTP, FTP, user-defined, etc.), a data store (files, DBs, etc.) or a person interacting through a text-based interface. [weight: 2]
- A **complex actor** is a person interacting through a graphical user interface (GUI). [weight: 3]

12/01/2026

47

47

## Use Case Point: UseCases



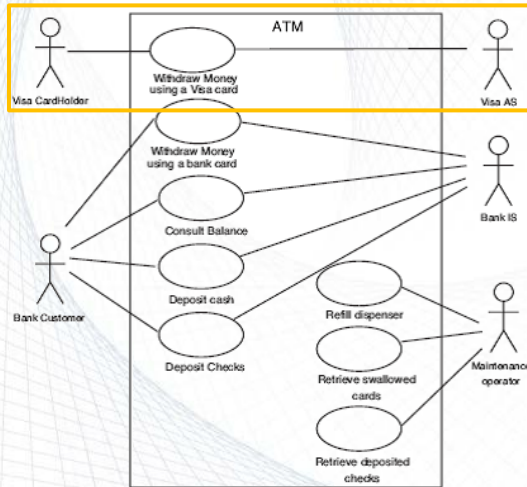
- For each Use Case, determine whether it is simple, average or complex.
- A **simple use case** contains up to 3 transactions. [weight: 5]
- An **average use case** contains 4-7 transactions. [weight: 10]
- A **complex use case** contains more than 7 transactions. [weight: 15]
- **Note:** The original Use Case Point method suggested to count only transactions of the main (success) scenarios. Some researchers propose to include in the counting also alternate (extensions) and exception (error) transaction sequences.

12/01/2026

48

48

## Example: ATM



### Actors:

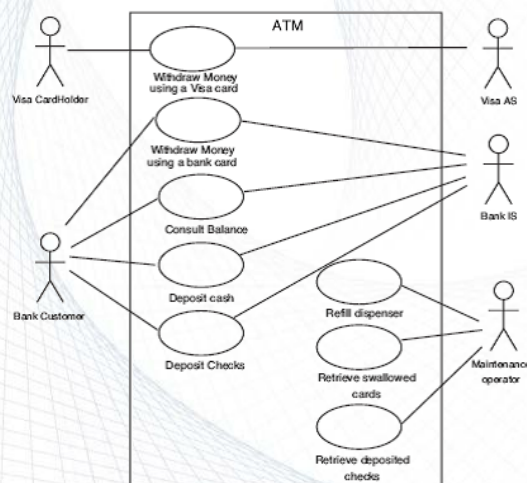
- Visa Cardholder: complex (interacts via GUI)
- Visa AS: average (interacts via communication protocol)

12/01/2026

49

49

## Example: ATM



### Use Case:

- “Withdraw money using a Visa card” complex (more than 7 transactions, even without counting extensions or exceptions)

■  $UAW = 3 + 2 = 5$

■  $UUCW = 15$

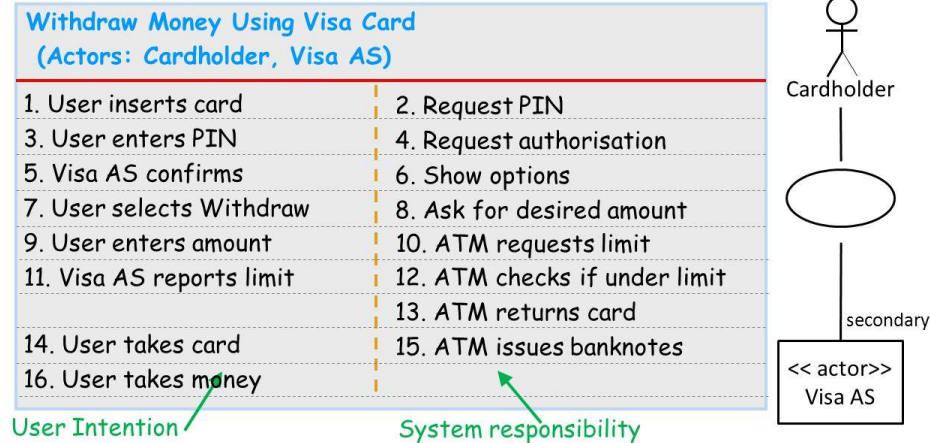
12/01/2026

50

50

## ATM: Use Case Body: Withdraw Money (Visa)

ENGR 110 10:15



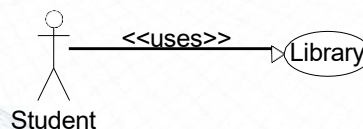
12/01/2026

51

51

## Use-Case Point /2

- We must count the inputs, outputs, files and data inquiries from use-cases.
- Function points become evident using the **use-case definition** and **activity diagram** for the use-case. Each step within the activity diagram can be a transaction (inputs, outputs, files and data inquiries).



12/01/2026

52

52

## Examining Use Case Scenario

- Use Case might be further refined via Activity and Interaction Diagrams
- Examining Use Case definition and Activity and/or Interaction diagrams can also lead to FP count.
- How?

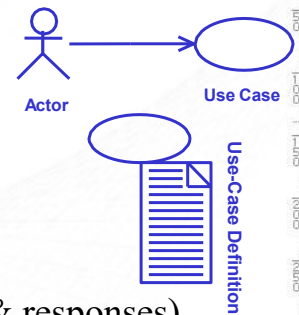
12/01/2026

53

53

## Use-Case Definition

- A typical *use-case definition* document consists of:
  - Use Case name
  - Actor name
  - Objective
  - Preconditions
  - Results (Post-conditions)
  - Detailed description (actions & responses)
  - Exceptions and alternative courses



12/01/2026

54

54

## Use-Case Definition Worksheet

| Use Case Definition       |                         |
|---------------------------|-------------------------|
| Last Updated by:          | Last Updated On:        |
| Use Case Name:            | UC ID:                  |
| Actor:                    |                         |
| Objective:                |                         |
| Preconditions:            |                         |
| Results (Postconditions): |                         |
| Detailed Description      |                         |
| Action                    | Model (System) Response |
| 1.                        | 1.                      |
| 2.                        | 2.                      |
| 3.                        | 3.                      |
| 4.                        | 4.                      |
| 5.                        | 5.                      |
| Exceptions:               |                         |
| Alternative Courses:      |                         |
| Original Author:          | Original Date:          |

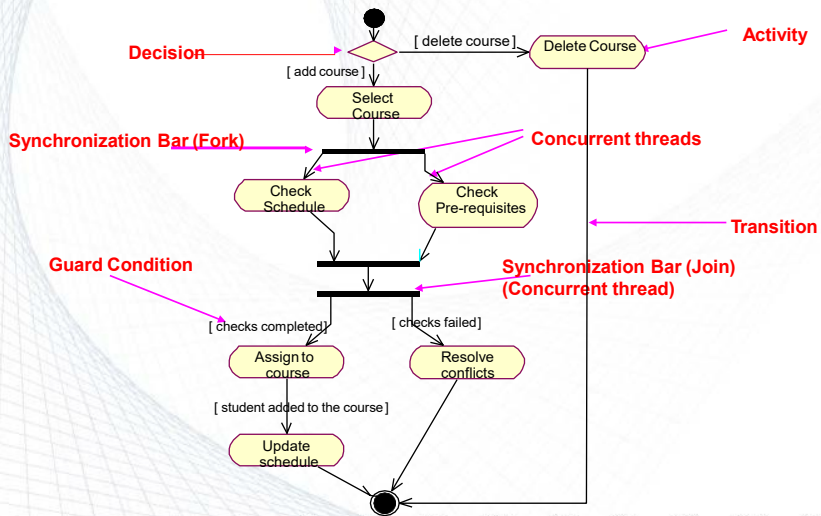
12/01/2026

55

55



## Example: Activity Diagram



12/01/2026

56

56



## Example 1: Use-Case Scenario

### Use-Case: Library Scenario:

1. The user may enter a book's ISBN number, student ID, or student name
2. The user will press "Find"
3. If the user enters a book ISBN number
  - The system will display information related to that book and write the results to a file
4. If the user entered a student name or student ID
  - The system will return a list of all books on the waiting list for that student and write the results to a file
  - The user can select one book from the list
  - The system will search the database by ISBN number
  - The system will display information related to that book and available date and will write the results to a file

Together they will  
qualify as 1 output

12/01/2026

57

57

## Example 1: Use-Case Scenario

### Use-Case: Library Scenario:

1. The user may enter a book's ISBN number, student ID, or student name (*input × 3*)
2. The user will press "Find" (*input × 1*)
3. If the user enters a book ISBN number (*input × 1*)
  - The system will display information related to that book and write the results to a file (*output × 1*) (*internal file × 1*)
4. If the user entered a student name or student ID (*input × 2*)
  - The system will return a list of all books on the waiting list for that student and write the results to a file (*output × 1*) (*internal file × 1*)
  - The user can select one book from the list (*external inquiry × 1*)
  - The system will search the database by ISBN number (*external file × 1*)
  - The system will display information related to that book and available date and will write the results to a file (*output × 1*) (*internal file × 1*)

But these are not independent from the "Find" input, so they are actually data attributes for "Find"

Together they will  
qualify as 1 output

12/01/2026

58

58

## Example 2 (work in team 3)

- Dr. X, a recent graduate from a medical university, is starting her medical practice in a small town. She is planning to hire a receptionist. She approaches software company Y to build a software system to manage the patients' appointments. The following is her problem description (next two slides).
- Suppose that you are the analyst in charge of estimating the cost of this system and you base your estimation of the function point (FP) count for this system.
- Calculate the unadjusted function point count (UFC) for the system assuming the average weight for all entries.

12/01/2026

59

59

## Example 2 (cont'd)

- When a patient calls for an appointment, the receptionist will ask the **patient's name** or **patient's ID number** and will **check the calendar** and will try to **schedule** the patient as early as possible to fill in vacancies. If the patient is happy with the proposed **appointment**, the receptionist will enter the appointment with the **patient name** and **purpose** of appointment. The system will verify the **patient name** and supply **supporting details** from the **patient records**, including the **patient's ID number**. After each appointment the Dr. X will **mark** the appointment as completed, **add comments**, and then schedule the patient for the next visit if

query

output

input

12/01/2026

60

60

## Example 2 (cont'd)

Note that we have to again ask ourselves whether these qualify for EI, EO, EQ, EIF and ILF.

| Type                 | Description                                                                                                          | No. | Weight | Weighted value |
|----------------------|----------------------------------------------------------------------------------------------------------------------|-----|--------|----------------|
| Inputs (EI)          | Patient name, Patient ID number, Appointment completed, Appointment purpose, Cancel appointment                      | 5   | 4      | 20             |
| Outputs (EO)         | Comments, Calendar, Supporting details, Appointment Information, Notification List, Daily schedule, Weekly schedule, | 7   | 5      | 35             |
| Queries (EQ)         | Check calendar, Query by name, Query by ID, Query by date, Verify patient, Available Appointment                     | 6   | 4      | 24             |
| Internal files (ILF) | Patients' data record                                                                                                | 1   | 10     | 10             |
| External files (EIF) | --                                                                                                                   | 0   | 7      | 0              |
| Total UFC            |                                                                                                                      |     |        | 89             |

12/01/2026

63

63

## Recent Approaches

- Recent approaches to size measurement and estimation include:
  - Decomposition
  - Expert opinion
  - Analogy
  - Class-code expansion

12/01/2026

64

64

## 4. Class-Code Expansion

- In the design phase, boundary classes are usually expanded by the ratio of 1 to 2 and control classes are expanded by the ratio of 2 to 3, respectively. The entity classes are usually expanded to a subsystem having 4 to 8 design classes.
- An average design class in Java has 20 methods, each method having 10 to 20 lines of code.

| <i>Analysis Classes Description</i>                                               |                |
|-----------------------------------------------------------------------------------|----------------|
|  | Boundary class |
|  | Control class  |
|  | Entity class   |

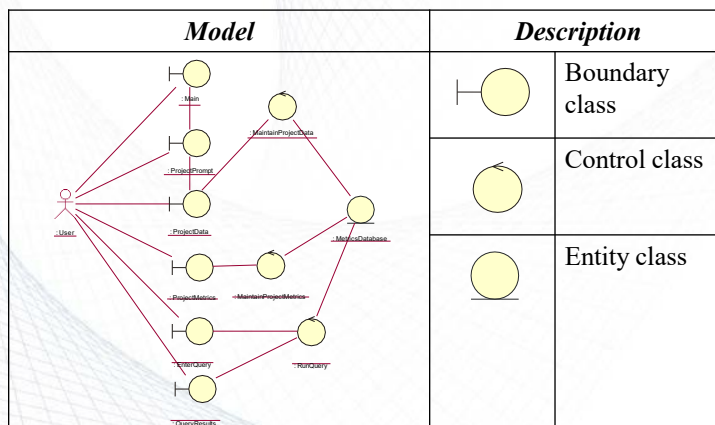
12/01/2026

65

65

## Example

- Using class-code expansion



12/01/2026

66

66



## Example (cont'd)

- Calculate total software size in LOC, assuming total size is  $1.5 \times$  methods LOC
- Calculate total development cost.

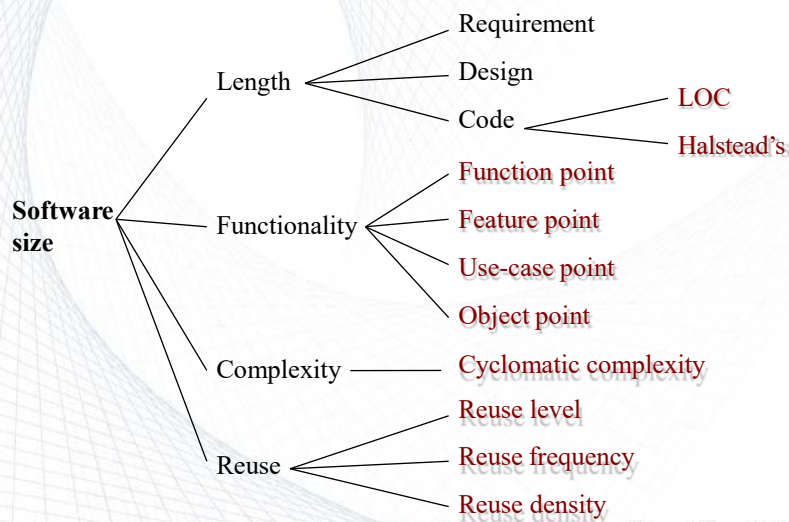
| <i>Analysis classes</i> | <i>No.</i> | <i>Total classes</i>           | <i>Total Operations</i>          | <i>Total LOC</i>                                                          |
|-------------------------|------------|--------------------------------|----------------------------------|---------------------------------------------------------------------------|
| <b>Boundary class</b>   | <b>6</b>   | <b>Min 16</b><br><b>Max 29</b> | <b>Min 320</b><br><b>Max 580</b> | <b><math>1.5 \times 320 \times 10 = 4,800</math></b><br><b>(minimum)</b>  |
| min ( $\times 1$ )      | 6          |                                |                                  |                                                                           |
| max ( $\times 2$ )      | 12         |                                |                                  | <b><math>1.5 \times 320 \times 20 = 9,600</math></b>                      |
| <b>Control class</b>    | <b>3</b>   |                                |                                  | <b><math>1.5 \times 580 \times 10 = 8,700</math></b>                      |
| min ( $\times 2$ )      | 6          |                                |                                  |                                                                           |
| max ( $\times 3$ )      | 9          |                                |                                  | <b><math>1.5 \times 580 \times 20 = 17,400</math></b><br><b>(maximum)</b> |
| <b>Entity class</b>     | <b>1</b>   |                                |                                  |                                                                           |
| min ( $\times 4$ )      | 4          |                                |                                  |                                                                           |
| max ( $\times 8$ )      | 8          |                                |                                  |                                                                           |

12/01/2026

67

67

## Software Size Metrics: Summary



12/01/2026

68

68



## Sizing Agile Projects Using Function Points

[https://www.youtube.com/watch?v=XI\\_iuFSow6A](https://www.youtube.com/watch?v=XI_iuFSow6A)

12/01/2026

69

69

- Albrecht, A. J., & Gaffney, J. E. (1983). Software function, source lines of code, and development effort prediction: a software science validation. *IEEE transactions on software engineering*, (6), 639-648.
- Function Points Analysis Training Manual (110 Pages)-  
<https://www.softwaremetrics.com/freemanual.htm>
- <http://softwarecost.org/tools/COCOMO/>
- <http://softwarecost.org/tools/COCOMO/>

12/01/2026

70

70

## Exercise 2: Use-Case Point Estimation

### Scenario:

A **Library Management System** is being developed. The system has:

- **Actors:**
  - **Librarian (GUI-based interaction)**
  - **Member (GUI-based interaction)**
  - **External Book Database (interacts via API)**
- **Use Cases:**
  - Borrow book (**Complex**, 8 transactions)
  - Return book (**Average**, 5 transactions)
  - Search book (**Simple**, 3 transactions)

12/01/2026

72

72

### Tasks:

1. Compute the **Unadjusted Use-Case Points (UUCP)** using the following weights:
  - **Actor Weight:**
    - Simple (API-based)
    - Average (protocol-based)
    - Complex (GUI-based) :
  - **Use-Case Weight:**
    - Simple :
    - Average :
    - Complex :
2. Calculate the **Adjusted Use-Case Points (UCP)** using a **VAF of 0.85**.

12/01/2026

73

73

## Suggested Solution:

**1. Compute Unadjusted Actor Weight (UAW):**

$$UAW = (1 \times 1) + (2 \times 3) = 7$$

**2. Compute Unadjusted Use-Case Weight (UUCW):**

$$UUCW = (1 \times 5) + (1 \times 10) + (1 \times 15) = 5 + 10 + 15 = 30$$

**3. Compute UUCP:**

$$UUCP = UAW + UUCW = 7 + 30 = 37$$

**4. Compute Final UCP:**

$$UCP = UUCP \times VAF = 37 \times 0.85 = 31.45$$

Thus, the **final Use-Case Points** is **31**.

12/01/2026

74

74

## Exercise 3: Object Point Estimation

**Scenario:** A team is developing an **Online Shopping System** that consists of the following elements:

- **Screens:**
  - Product catalog (Medium)
  - Shopping cart (Simple)
  - Checkout page (Difficult)
- **Reports:**
  - Order summary (Simple)
  - Sales report (Medium)
- **Third-generation language (3GL) components:** 1

**Tasks:**

1. Compute the **Object Points (OP)**.
2. Assume **10% reuse** from a previous project. Compute the **Adjusted OP**.
3. If developer productivity is **15 OP per person-month**, estimate the effort required.

12/01/2026

75

75

## Suggested Solution:

### 1. Compute Raw OP:

$$\begin{aligned} OP &= (1 \times 2) + (2 \times 1) + (1 \times 3) + (1 \times 2) + (1 \times 5) + (1 \times 10) \\ &= 2 + 2 + 3 + 2 + 5 + 10 = 24 \end{aligned}$$

### 2. Adjust OP for reuse (10% reused):

$$Op_{new} = OP \times (100 - r) / 100 = 24 \times 0.90 = 21.6$$

### 3. Estimate Effort:

$$\text{Effort} = OP_{new} / \text{Productivity} = 21.6 / 15 = 1.44 \text{ person-months}$$

Thus, **1.44 person-months** of effort is required to develop the system.

12/01/2026

76

76

## Project Title: Event Ticket Booking Application

### 1. System Requirements

#### System Description

The application allows users to **browse and purchase tickets for various events** such as concerts, conferences, and sports matches. Event organizers can **list their events**, and the system will handle ticket sales and payments. Admins will manage users and event listings.

Q1: List Actors, UC của ứng dụng

Q2: Vẽ UC diagrams

Q3: Phân tích độ phức tạp của Actor, UC

12/01/2026

77

77

### Customers, Event Organizers, Admins, and the System itself.

#### Example Use Cases:

1. Register & Login
2. Search for Events
3. Purchase Tickets
4. Make Payment
5. Generate Digital Ticket
6. Manage Events (Organizers)
7. Admin Approves Events

12/01/2026

78

78

### Determine Use Case Complexity

For each **use case**, determine its complexity based on the following criteria:

| Use Case Type       | Simple (5 Points) | Average (10 Points)   | Complex (15 Points)    |
|---------------------|-------------------|-----------------------|------------------------|
| Transaction Steps   | ≤ 3 steps         | 4 - 7 steps           | > 7 steps              |
| Entities Involved   | 1 - 2 entities    | 3 - 4 entities        | > 4 entities           |
| System Interactions | Basic CRUD        | Involves calculations | Complex business logic |

Use this table to categorize your use cases as **Simple, Average, or Complex** and assign points accordingly.

12/01/2026

79

79